Follow view and the two cores

The glass shows the active zone. The knob sets volume. Touch replaces the missing dial click. The picture is a uimk2 zlist. LVGL is not on this path.

This file is pseudocode for what runs today. Layout is one firmware file: `firmware/waveshare-knob-touch-lcd-1.` (API `follow_glass.h`). Core B host is `uimk2_host` on the event ring. Anything not described here as running is out of scope or parked at the end.

Core A is UI only: touch, rotary count and pooled rotation clicks, and pushing the panel. Everything else is Core B, including the player.

Constants

Named in `uimk2_config.h`. Changing a value does not change control flow.

Name	Meaning	Current Waveshare setting
PANEL_W	framebuffer width in pixels	360
PANEL_H	framebuffer height in pixels	360
QUEUE_DEPTH	event-ring slots	8
CLOCK_RES	one tick of the absolute clock	1 ms
CLOCKS_PER_SEC	ticks in one logical second	1000
POLL_EVERY_START	poll gap armed on wake	1 s
POLL_EVERY_MAX	poll-gap cap	16 s
SCREEN_OFF_EVERY	backlight-off delay	30 s
UPDATE_RENDER_EVERY	update + render_draw + clock cadence while awake	200 ms
REDRAW_RETRY_EVERY	delay before a busy redraw is retried	50 ms
REDRAW_ATTEMPT_CAP	busy redraws before ERROR	5
HIT_OPAQUE_PCT	min opacity percent for an opaque hit (0 = off)	0
SCROLL_MS_PER_PX	ms per px of SCROLL ox (smaller = faster)	50

last_run, every, and expire_by count CLOCK_RES ticks. Each framebuffer is PANEL_W * PANEL_H pack66 words (same byte size as RGBA8888). The dirty bitmap is PANEL_H bits, one per row, filled only by comparing curr to next.

The glass (follow_glass.cpp)

Waveshare ESP32-S3 knob, round bezel. No hardware push-click. Player today is Volumio HTTP. A detent changes volume. Touch on the transport button toggles play/stop. The only panel-sized preallocated pixmap is the base shapes px (360x360 opaque RGBA; boot: Disintegration; albumart ROM-TJpgDec NN overwrites it). That buffer is not a side channel: it is shape index 8 (FOLLOW_BASE, is_base) in the zlist. Text/digit shapes keep their own smaller baked maps; those are not a second panel pixmap.

```
+----------------------+
|      play / stop     |   y = 8, centered
+----------------------+   wider of the two labels, pad 12 x 6
HH:MM:SS                   CLOCKTEXT, under the button
spinme                     x = 24, yellow ink
Artist                     white ink
Album
long title...              SCROLL when map wider than viewport
010                        three digits
011                        desired, same box, alpha 128, only
                           while it differs from the polled digits
```

Zlist index 0 = closest. Build order of indices:

```
0 transport (FIXED, accept_hit, is_transport)
1 desire    (FIXED)
2 volume    (FIXED)
3 clock     (CLOCKTEXT)
4 title     (SCROLL or FIXED)
5 album     (SCROLL or FIXED)
6 artist    (SCROLL or FIXED)
7 zone      (SCROLL or FIXED)
8 base      (is_base; sole panel pixmap px, pack66 into next)
```

SCROLL: ox = (clock_tick / SCROLL_MS_PER_OX) % pw. When the ink is wider than the viewport, the bake string gets three trailing spaces so the wrap seam has a gap. CLOCKTEXT: rewrite shape.text to 24h HH:MM:SS from clock_tick (millis). Raster (follow_glass_raster) bakes glyph lines into RGBA maps; update does not write pixels.

Albumart fill of FOLLOW_BASE.px

On getState completion (non-blocking HTTP), if `albumart` changed: queue a GET of that relative URL, buffer JPEG in PSRAM, then a side task runs ROM TJpgDec (`jd_prepare` / `jd_decomp`) into FOLLOW_BASE.px.

Fill strategy (`rom_jpeg_decode_nn_rgba`):

1. Clear the whole base to opaque black (A=255).
2. After prepare, pick the **coarsest** ROM scale (0=1/1 .. 3=1/8) whose output size is still >= PANEL on both axes; else scale 0. That is the `via` size (e.g. 1000x1000 -> scale 1/2 -> 500x500).
3. Each MCU callback is an RGB888 rectangle in that scaled space. Dest pixels use integer NN: `sx = dx * out_w / dst_w`, `sy = dy * out_h / dst_h`. An MCU owns every dest whose sample lands in its source rect (spans via ceil so adjacent MCUs abut no black grid lines).
4. Write R,G,B and force A=255. Mark FOLLOW_BASE dirty; next URD packs it.

Zlist shape (what paint uses)

Pixels in shape `px` are straight-alpha RGBA8888, R G B A, row-major. Frame buffers after paint are pack66 (RGB666 + A2; A2==0 is a skip run). A sample outside the map is transparent (skip). `(0,0,0,0)` in the map is transparent paper.

```
shape:
  update_selector   FIXED | SCROLL | CLOCKTEXT
  dirty, is_base, accept_hit, is_transport
  x, y, w, h        exterior / viewport on the panel (sealed at build)
  pw, ph            text-map size (0 => w, h)
  ox, oy            SCROLL origin into px
  px                RGBA source, or NULL => solid rgba[4]
  rgba[4]
  text[]            CLOCKTEXT string
  child[]           optional; follow_glass leaves nchild = 0
```

Update / event / scheduler dispatch is a switch on a small int selector. Invalidate is not an event.

How a frame is built

```
follow_glass_build:
  alloc FOLLOW_BASE.px (sole panel pixmap); seed Disintegration
  measure glyphs; alloc per-line text maps; place descs; zlist_build
  follow_glass_raster(...)

follow_glass_raster(polled, desired, playing, desired_on):
```

```
  paint_text_map zone/artist/album/title (SCROLL if pw > view)
  paint_text_map clock from shape.text or "00:00:00"
  paint volume digits; copy to desire; if desired_on, blend cyan @ 128
  paint transport label play|stop on white paper
  # base.px updated on albumart poll (ROM jpeg NN), not here
```

Update then render

```
update(shape):
  switch shape.update_selector:
    FIXED:     return
    SCROLL:    ox = (z.clock_tick / SCROLL_MS_PER_OX) % pw; dirty if changed
    CLOCKTEXT: text = HH:MM:SS from clock_tick; dirty if text changed

update_render_draw():
  for shape in zlist: update(shape)
  for root in roots: mark_dirty_from_children(root)   # no-op if nchild=0
  for i from n-1 down to 0:           # farthest first
    if is_base: pack each row of FOLLOW_BASE.px into next (pack66)
    else:       viewport-sample px (SCROLL wraps mod pw), src-over66 into next
    shape.dirty = false
  if busy-by-core-A: arm redraw retry; return
  compare curr and next; set row dirty bits
  if any bit set:
    swap curr and next
    busy-by-core-A = 1
    ping A.redraw()
```

No clear of next first. No memmove of scrolled pixels. Hit-test: index 0 upward, first `accept_hit` whose exterior contains the point (HIT_OPAQUE_PCT 0 skips the opacity sample).

Two cores

```
encoder IRQ ----+
touch IRQ ------+--> Core A ---- posts event values ----> Core B ring
                |         ^                                  |
                |         | busy-by-core-A                   | host_run
                |         | dirty-line bits                  | host_step
                |         +---- panel bus <--- curr          | URD into next
                                        next belongs only to B
```

Core A: decode press and rotary; post press / volume / screen_on / screen_off; run redraw (push dirty

rows of curr, clear busy). A does not know play/pause, Toggle, desired, or the player. A never sees next.

Core B owns three cooperating pieces: the **scheduler**, the **event ring**, and the **player HTTP state machine**. Only the ring is the cross-core channel. The HTTP machine never pushes onto the ring. JPEG decode is a fourth actor (short-lived side task) that only dirties FOLLOW_BASE when done.

Scheduler, event queue, and HTTP state machine

Event queue (`uimk2_ring`)

Ring of QUEUE_DEPTH value slots. Peek, handle, then pop. Full ring: ERROR, drop the new event. Handlers update host / zlist state and may **arm** player IO; they must not block on sockets or DNS.

```
handle(event):
  switch selector:
    press:              hit-test (x, y); transport => Toggle (+ maybe Poll)
    volume:             desired = new_val; queue volume cmd when burst ends
    toggle:             flip local playing; queue toggle cmd
    screen_on:          backlight on; arm poll + screen-off; push URD
    screen_off:         backlight off; disarm poll
    update_render_draw: set clock_tick; update_render_draw()
    redraw:             retry compare/swap/ping
    clock:              clock_tick = now; zlist_set_clock (no paint alone)
    player_poll:        queue non-blocking getState
```

A posts: press, volume, screen_on, screen_off. Each post wakes B (task notify). Scheduler posts: URD, CLOCK, PLAYER_POLL, SCREEN_OFF, REDRAW retry.

SCROLL / CLOCKTEXT pixels move only when URD runs. CLOCK alone does not paint.

Schedulers (`uimk2_host`)

Soft timers: each slot has `last_run` and `every` (CLOCK_RES ms). Due slots push **one ring event each**; they never touch sockets.

Permanent (`perm` list):

- update_render_draw every UPDATE_RENDER_EVERY while backlight on
- clock every UPDATE_RENDER_EVERY (same cadence; redundant with URD tick copy)
- player_poll POLL_EVERY_START on wake, doubles to POLL_EVERY_MAX while armed

One-offs (packed, zero-terminated): screen_off at SCREEN_OFF_EVERY; redraw retry at REDRAW_RETRY_EVERY when busy (ERROR at REDRAW_ATTEMPT_CAP).

`host_run(now)` fires due slots, then returns ms until the next due time. URD is gated on backlight; poll on `poll_enabled`. Idle screen-off therefore stops visible clock/scroll even if CLOCK keeps enqueueing.

HTTP state machine (`player_http`)

Not on the ring. One in-flight socket op. Pending flags coalesce: getState while busy sets a flag; latest volume wins; toggle/albumart queue.

Phases: idle -> connect -> send -> recv headers -> recv body -> idle. Advanced only by `player_http_tick()` with `select` timeout 0. Boot may block once for WiFi + Volumio DNS; runtime never calls `hostByName`.

Completions are **return codes from tick**, not ring events:

- getState OK -> refresh track/volume/playing; maybe queue albumart if URL changed
- albumart OK -> hand JPEG to a decode side task (Core B stays free)
- cmd OK/FAIL -> log; no URD stall

How they meet (Core B loop)

```
B loop:
  now = millis()
  delay = host_run(now)              # scheduler -> ring only
  while peek:
    bake raster if needed
    host_step                        # ring handler; may queue player IO only
  ev = player_http_tick()            # select(0); never blocks
  apply ev (snapshot / queue albumart / spawn decode) if any
  if http busy: delay = 0
  sleep interruptible up to min(delay, 50ms)   # notify from A wakes early


A.redraw():
  for each dirty row: push that row of curr (bezel clip)
  clear bits; busy-by-core-A = 0
```

Contract: scheduler produces cadence on the ring; ring handlers only arm the HTTP machine; the machine runs cooperatively beside the scheduler and must not stall URD. Blocking GET/DNS on Core B is a bug.

Sample timeline

Start: screen off, ring empty, busy clear.

Detent at t=0: A posts screen_on + volume(41). B arms poll and screen-off, sets desired, URD paints, redraw swaps and pings A. Volume cmd is queued on the HTTP machine; tick completes it without blocking the next URD.

While getState is in flight, A may post volume(42), volume(43) on the ring; handlers run immediately (overlay updates). The HTTP machine keeps the latest volume pending behind the in-flight op. Overlay is desire @ alpha 128 over polled digits.

Title wider than viewport: SCROLL updates ox from clock_tick inside URD; no scroll event on the ring.

Snapshot while backlight off updates next and does not ping A.

Blits (running)

- Shape `px`: RGBA8888. Frames: pack66.
- Transparent A2==0: skip-RLE in the low 24 bits; encode_row_skips after pack.
- Paint: src_over66 into next. ESP32-S3: PIE for aligned opaque spans (`uimk2_blit_pie.S`).
- Core A unpacks pack66 rows to RGBA for the panel bus.

Host library (not the live glass)

`uimk2_bitmap` / `uimk2_line` (Layout, L2R, Glyph, Cache, Viewport) exist for host tests and glyph tooling. The device glass does not build that tree. Cache is not an active compose path on device.

Out of scope

Greenthreads. LVGL. Function pointers for dispatch. Memmove of a scrolled line. Selective damage into next as the primary path. Invalidate as an event. Recursive tree walk for update/render on the glass. Software JPEG (ROM TJpgDec is in use). MPD. Sonos. Audio effects.

Parked (not owed)

Further SWAR/PIE ideas on todo:c029d7c6.